

id-qualifier:

tag
optional

STRUCTURE declares an element that is encoded as a TLV **Structure**. The **STRUCTURE** fields SHALL be separated by commas.

A **STRUCTURE** definition declares the list of fields that MAY appear within the corresponding TLV **Structure**. Each field definition gives the type of the field, its tag, and an associated textual name. The field type MAY be either a fundamental type, a **CHOICE OF** pseudo-type, an **ANY** pseudo-type, or a reference to one of these types defined outside the **STRUCTURE** definition.

A **STRUCTURE** definition MAY also contain one or more **includes** statements. Each such statement identifies a **FIELD GROUP** definition whose fields are to be included within the TLV **Structure** as if they had been declared within the **STRUCTURE** definition itself (see **Includes FIELD GROUP** below).

An **extensible** qualifier MAY be used to declare that a structure can be extended at encoding time by the inclusion of fields not listed in the **STRUCTURE** definition.

The **order** qualifiers (**any-order**, **schema-order** and **tag-order**) MAY be used to specify a particular order for the encoding of fields within a TLV **Structure**.

A **nullable** qualifier MAY be used to indicate that a TLV **Null** MAY be encoded in place of the **STRUCTURE**.

B.3.9.1. Fields

Fields within a **STRUCTURE** are assigned textual names to distinguish them from one another. Each such name SHALL be distinct from all other field names defined within the **STRUCTURE** or included via a **includes** statement. Fields names do not affect the encoding of the resultant TLV, but MAY serve as either user documentation or input to code generation tools.

Per the rules of TLV, all fields within a TLV **Structure** SHALL be encoded with a distinct TLV tag. Field tags are declared by placing a **tag** qualifier on the field name. Both protocol-specific and context-specific tags are allowed on the fields in a **STRUCTURE** definition.

For a given field if the tag qualifier is missing then the underlying type SHALL provide a **default tag**. This can occur in two situations:

1. the underlying type is a reference to a type definition that provides a default tag
2. the underlying type is a **CHOICE OF** pseudo-type whose alternates provide default tags.

The tags associated with **includes** fields are inherited from the target **FIELD GROUP** definition.

All tags associated with the fields of a TLV **Structure** SHALL be unique. This is true not only for tags declared directly within the **STRUCTURE** definition, but also for any tags associated with fields that are incorporated via an **includes** statement.

The **anonymous** tag SHALL NOT be used as the tag for a field within a **STRUCTURE** definition.